

2. Control PWM Servo

2. Control PWM Servo

- 2.1 Experiment Objective
- 2.2 Experiment Preparation
- 2.3 Experiment Procedure
- 2.4 Experiment Summary

2.1 Experiment Objective

This course focuses on learning how to control the functionality of the robot's servo gimbal.

Note: As the RS version of the robot does not come with a servo gimbal by default, you will need to provide your own PWM servo gimbal to follow this tutorial.

2.2 Experiment Preparation

This course involves the following function from the Muto hexapod robot's Python library:

Gimbal_1_2(S1, S2): Controls the servo gimbal. The parameters S1/S2 represent the angle values of the PWM servos S1/S2. The valid range for S1 is [0, 180], and for S2 is [0, 115].

When one of the values for S1/S2 is -1, it indicates that the corresponding PWM servo will not be operated. If you only need to control the S1 servo without changing the position of the S2 servo, you can pass -1 for S2.

PWM servo gimbal wiring instructions:

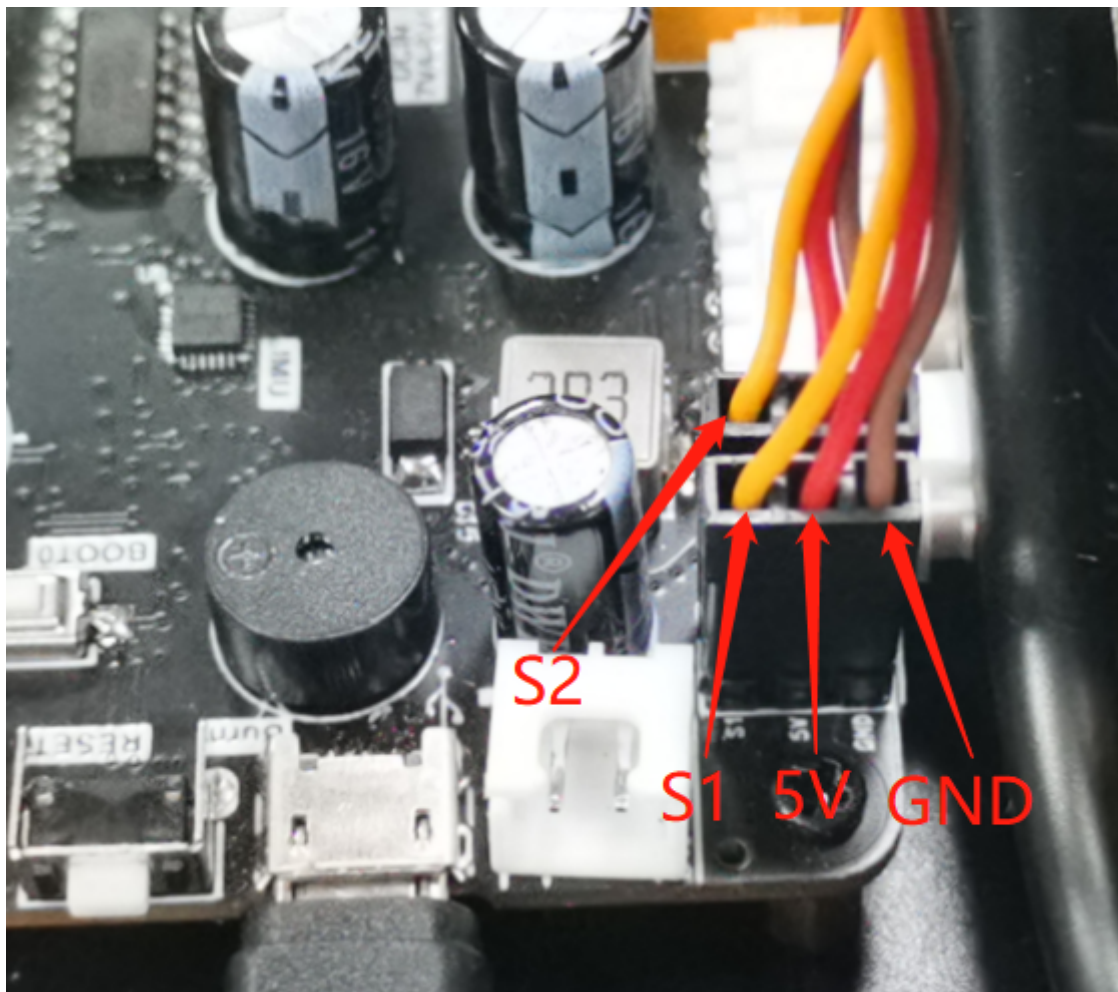
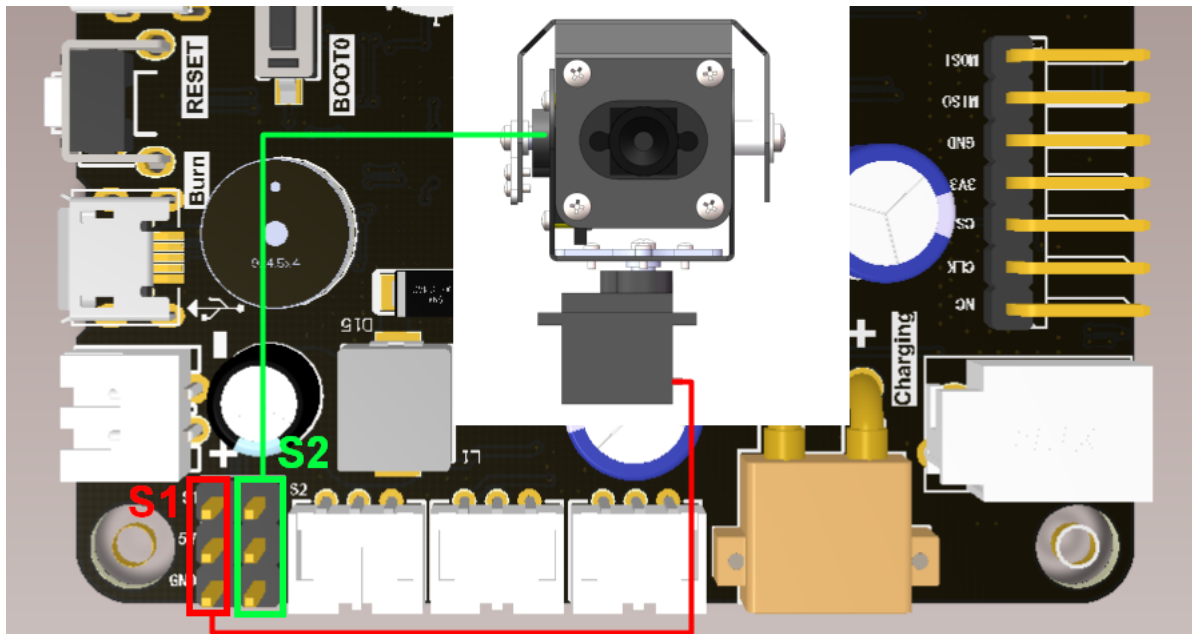
Lower servo connects to S1

Upper servo connects to S2

Black wire plugs into GND

Red wire plugs into 5V

Orange wire plugs into S1/S2



2.3 Experiment Procedure

Open the JupyterLab client and navigate to the code path:

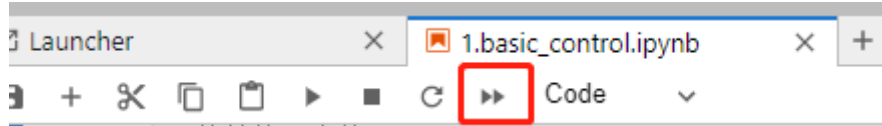
```
muto/samples/Control/2.pwm_servo.ipynb
```

By default, `g_ENABLE_CHINESE` is `False`. If you need to display Chinese, please set `g_ENABLE_CHINESE=True`.

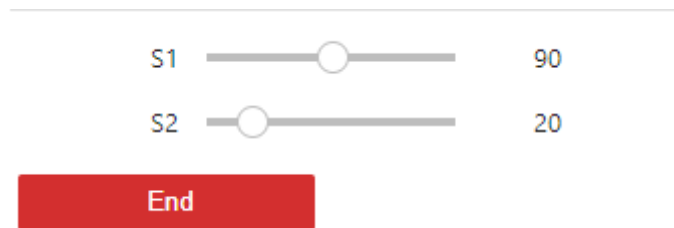
```
# Chinese switch. The default value is English
g_ENABLE_CHINESE = False

Name_widgets = {
    'End': ("End", "Finish")
}
```

Click 'Run All Cells', then scroll to the bottom to see the generated controls.



Operate the different controls to manage their respective functions.

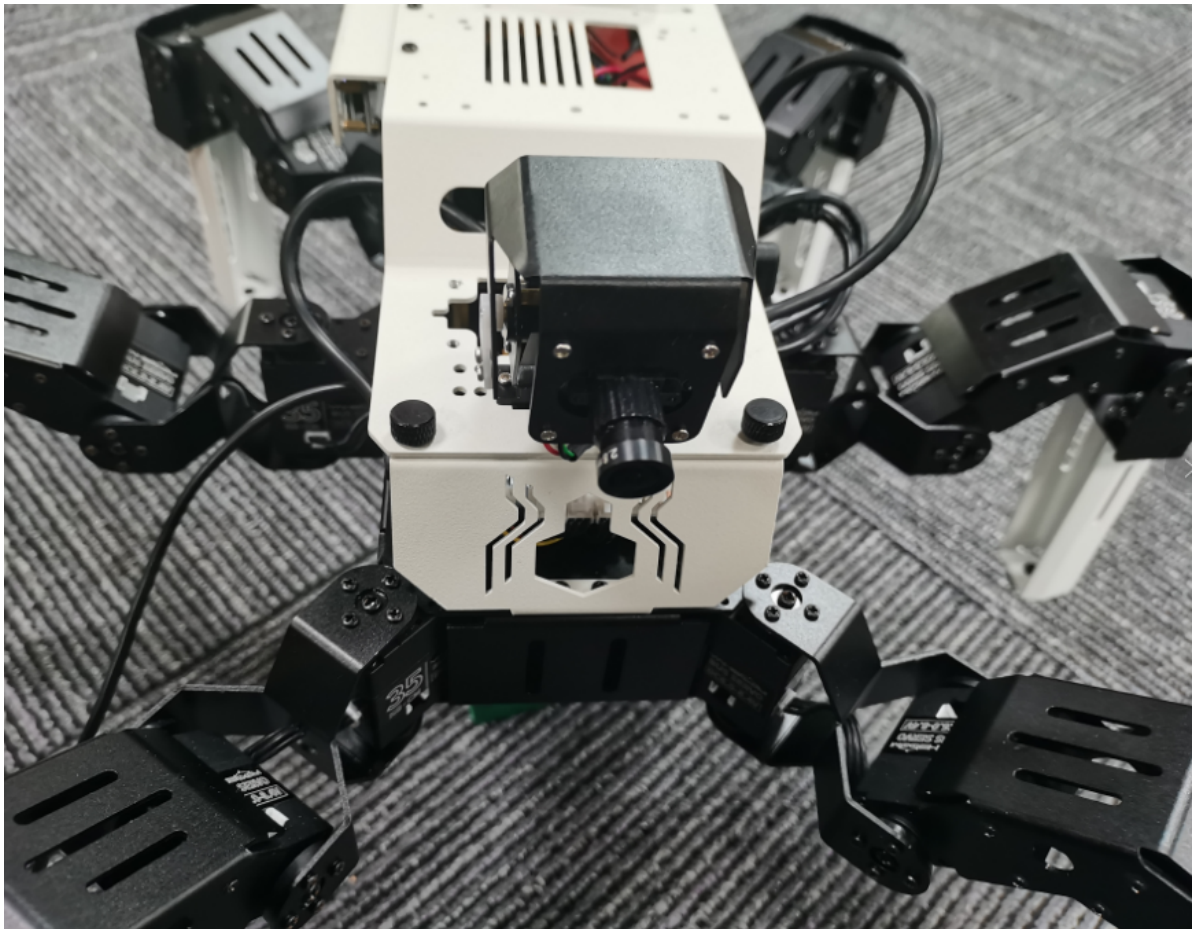


Each time you slide the S1 slider, the angle of the robot's S1 servo changes immediately, controlling the left-right rotation of the camera on the gimbal. Similarly, each time you slide the S2 slider, the angle of the robot's S2 servo changes, controlling the up-down movement of the camera on the gimbal.

```
def pwm_servo(S1, S2):
    g_bot.Gimbal_1_2(S1, S2)
    return S1, S2
```

2.4 Experiment Summary

In this experiment, we used JupyterLab controls to manage the hexapod robot's servo gimbal. By simply dragging the S1/S2 sliders, you can easily adjust the area captured by the camera on the gimbal.



To exit the program, press the 'End' button. The program will automatically reset the servo gimbal to its initial position.

Note: Although the camera is mentioned in this example, it is not actually driven by the program. This is for functional illustration only. Subsequent courses will include examples of practical applications driving both the servo and the camera.